

VFSTR R22 ACADEMIC REGULATIONS RAG ASSISTANT

Project Manual / Technical Module — B.Tech Artificial Intelligence & Machine Learning

Student Name	K. Abdul Shukkur
Registration No.	231FA18067
Program / Section	B.Tech AI & ML — Section C
Academic Year	Final Year — 4-1
Project Type	RAG-based Academic Regulation Question Answering System

1. Abstract

The VFSTR R22 Academic Regulations RAG Assistant is a document-grounded question-answering application developed to help students retrieve academic regulation information quickly. The system uses Retrieval-Augmented Generation (RAG), where information is first retrieved from the VFSTR B.Tech R22 Academic Regulations and then supplied as context to a generative AI model. The application uses PyPDF for document extraction, Recursive Character Text Splitter for chunking, the all-MiniLM-L6-v2 Sentence Transformer for embeddings, FAISS for vector similarity search, Google Gemini 3.6 Flash for answer generation, and Streamlit for the user interface. The assistant also displays the relevant source pages and is instructed not to invent information outside the retrieved regulation context.

2. Problem Statement

Academic regulations are often available as lengthy PDF documents containing many rules related to attendance, credits, assessment, progression and academic requirements. Students may spend considerable time manually searching these documents for individual questions. The proposed system provides a conversational interface that retrieves relevant sections from the R22 regulations and presents the information in a concise form.

3. Objectives

- Develop a focused academic question-answering assistant for VFSTR R22 regulations.
- Convert the regulations PDF into searchable semantic representations.
- Retrieve relevant document sections using vector similarity search.
- Generate context-grounded answers using Google Gemini.
- Display source page information so users can verify the original regulation.
- Reduce unsupported or hallucinated responses by restricting generation to retrieved context.

4. Technologies and Their Roles

Technology	Role in the Project
Python	Core implementation language.
PyPDF	Loads the 40-page PDF and extracts text.
LangChain Text Splitter	Creates overlapping text chunks for retrieval.
Sentence Transformers	Creates 384-dimensional semantic embeddings using all-MiniLM-L6-v2.
FAISS	Indexes embeddings and retrieves the closest chunks.
Google Gemini 3.6 Flash	Generates the final answer from retrieved context.
Streamlit	Provides the interactive browser-based interface.
python-dotenv	Loads the API key securely from the local .env file.

5. System Architecture and Workflow

INPUT	VFSTR R22 Academic Regulations PDF
↓	
PROCESS 1	Text extraction using PyPDF
↓	
PROCESS 2	Chunking: 1,000-character chunks with 200-character overlap
↓	
PROCESS 3	Embedding generation using all-MiniLM-L6-v2
↓	
DATABASE	FAISS vector index
↓	
USER QUERY	Question entered through Streamlit
↓	
RETRIEVAL	Query embedding → top 3 relevant chunks from FAISS
↓	
GENERATION	Retrieved context + question → Google Gemini 3.6 Flash
↓	
OUTPUT	Concise answer + relevant VFSTR R22 source pages

6. Implementation Details

Document processing: The application reads the 40-page VFSTR R22 regulations PDF and extracts approximately 108,521 characters. The extracted text is split into 138 overlapping chunks. Overlap helps preserve context when a regulation statement crosses a chunk boundary.

Semantic retrieval: Each chunk is encoded using the all-MiniLM-L6-v2 Sentence Transformer. The resulting vectors are stored in a FAISS IndexFlatL2 index. A user's question is encoded using the same embedding model, and the three closest vectors are retrieved.

Answer generation: The retrieved chunks are inserted into a controlled prompt. Gemini is instructed to use only this context, avoid inventing facts, and explicitly state when the requested information cannot be found in the VFSTR R22 regulations.

Performance optimization: The Streamlit application uses resource caching so the PDF, embedding model, FAISS index and Gemini client are not unnecessarily recreated for every question during a session.

7. Sample Test Case and Result

Test Item	Observed Result
Question	What are the attendance requirements for students?
Retrieved information	Attendance expectations, 75% minimum course attendance, and condonation provisions.
Generated response	The system produced a concise answer based on the retrieved regulation context.
Source display	Relevant VFSTR R22 page numbers were displayed.

8. Key Features

- Semantic search: retrieves information based on meaning rather than only exact keyword matching.
- Grounded generation: Gemini receives retrieved regulation content as context.
- Source transparency: relevant document pages are displayed with the answer.
- Out-of-scope handling: the prompt instructs the assistant to state when information is not found in the regulations.
- Simple deployment: the application runs locally through Streamlit.

9. Advantages

- Faster access to information from a lengthy academic regulations document.
- Easy-to-use conversational interface for students.
- Semantic retrieval improves matching for differently worded questions.
- Source pages help users verify information against the original document.
- Modular architecture allows the knowledge base to be expanded later.

10. Limitations

- The current knowledge base is limited to the VFSTR R22 Academic Regulations PDF.
- The quality of the final answer depends on PDF text extraction and retrieval quality.
- The application requires access to a valid Google Gemini API key.
- It is an academic information assistant and should not replace official university notifications or administrative decisions.

11. Future Scope

- Add multiple regulations, circulars, examination rules and department documents.
- Add conversation memory for follow-up questions.
- Show the exact retrieved passages alongside source pages.
- Add authentication and deploy the application for controlled student access.
- Add multilingual question answering.
- Introduce document versioning so answers can be associated with a specific regulation year.

12. Learning Outcomes

- Understanding of Retrieval-Augmented Generation architecture.
- Practical use of text embeddings and vector similarity search.
- Experience with FAISS-based information retrieval.
- Integration of a generative AI model into a Python application.
- Development of an interactive Streamlit application.
- Understanding of responsible document-grounded question answering.

13. Conclusion

The VFSTR R22 Academic Regulations RAG Assistant successfully demonstrates the integration of document processing, semantic retrieval, vector databases and generative AI in a focused academic application. The system converts the R22 regulations into searchable semantic representations, retrieves relevant sections for a user question, and generates a concise response using Gemini. The Streamlit interface and source-page display make the system practical for demonstrating document-grounded AI in an academic setting.

14. Project Demonstration Flow

Step 1	Start the application with <code>`streamlit run app.py`</code> .
Step 2	Enter an academic regulation question.
Step 3	System converts the question into an embedding.
Step 4	FAISS retrieves the three most relevant document chunks.
Step 5	Gemini generates an answer using the retrieved context.
Step 6	Answer and source pages are displayed in the browser.

Keywords: Retrieval-Augmented Generation, RAG, Semantic Search, Embeddings, FAISS, Sentence Transformers, Generative AI, Gemini, Streamlit, Academic Regulations, Question Answering.